

Documentación técnica

Sokoban

TPO Procesos de desarrollo de software

Integrantes:

- Santiago González
- Julian Ezequiel Isnardo
- Facundo Llano
- Joaquín Zampini Quiroga
- Augusto Pugliano
- Francisco Quinteros

Última actualización: 27/5/2026

Índice

Índice.....	2
Objetivos y alcance del proyecto.....	2
Elementos a hacer:.....	2
Casilleros intermedios.....	4
Ideas de funcionalidades (2 a elegir).....	6
Requerimientos funcionales:.....	7
Requerimientos no funcionales:.....	9
Métodos y estrategias:.....	9
Jerarquía de Clases y Arquitectura del Modelo.....	10
Clase base: Entidad.....	10
Clasificación de Entidades.....	11
Implementación de casilleros intermedios.....	11
Sistema de canales para llaves/cerrojos/muros.....	12

Objetivos y alcance del proyecto

Proyecto universitario que pretende desarrollar un juego tipo Sokoban usando un correcto uso de los patrones de diseño y evitando caer en code smells, el juego aparte de los requisitos base debe tener 2 funcionalidades extras (aún no definidas) y una estética determinada (tampoco definida)

El juego será visto de forma cenital, en 2D. Se renderiza en java swing.
Los controles serán las flechas del teclado o WASD

Elementos a hacer:

- **Pared:** Bloquea el paso
- **Piso:** Permite el paso
- **Sokoban:** Personaje jugable
- **Caja Normal:** Caja normal
- **Caja llave:** Caja que debe ser transportada hasta la casilla candado
- **Casilla candado:** Al tener encima la caja llave, tooglea las paredes bloqueadas a abiertas
- **Destino de las cajas:** Donde se depositan las cajas para ganar
- **Caja frágil:** Caja que tiene límite de desplazamientos
- **Pared bloqueada:** Bloquea el paso hasta que esté abierta por la casilla candado
- **Pared abierta:** Pared que estaba bloqueada y ahora abierta permite el paso
- **Piso resbaladizo:** Sokoban y las cajas que pasen por acá se desplazan hasta el fondo

Caracter	Entidad	Descripción
P	Pared	Bloque estático que delimita el mapa y no puede ser atravesado.
S	Suelo	Espacio vacío por el cual el jugador y las cajas pueden moverse libremente.
J	Jugador	Representa la posición inicial del personaje principal (Sokoban).
C	Caja normal	Bloque estándar que el jugador debe empujar hacia los destinos.
F	Caja frágil	Caja con una cantidad limitada de movimientos antes de romperse.
L-[CANAL]	Caja llave	Identificada por canal (ej: L-AZUL, L-ROJA). Solo activa cerrojos del mismo canal al estar posicionada en el cerrojo pertinente.
L-[CANAL1]+[CANAL2]	Caja llave multicanal	Ejemplo de caja llave con 2 o más canales, la idea es que si tienes 1 cerrojo azul, uno rojo, y una caja rojo-azul solo puedes tener un muro abierto a la vez (Es la misma caja llave no es otra entidad, solo es un ejemplo de como funcionaria con 2+ canales)

D	Casilla destino	Ubicación donde deben colocarse las cajas para ganar el nivel.
R	Terreno resbaladizo	Superficie que hace que las cajas se deslicen hasta encontrar un tope.
X-[CANAL]	Casillero cerrojo	Cerrojo identificado por canal. Se activa con llave del mismo canal. No desactiva otros canales
X-[CANAL]-EXC	Cerrojo exclusivo	Cerrojo que al activarse con llave multicanal, desactiva los otros canales de esa llave.
M-[CANAL]	Muro cerrado	Puerta bloqueada por canal. Se abre cuando todos los cerrojos del canal están activados.
A-[CANAL]	Muro abierto	Estado transitable del muro tras activación de su canal.

Casilleros intermedios

En pos de no perder información, no basta con tener el casillero de piso especial (Destino, cerrojo,...) y los objetos que se pueden parar encima (Sokoban, Caja, Caja frágil,...) Se debe guardar una combinación intermedia de ambas entidades para que no se pierda información. Por ejemplo dejas la caja en el destino y quieres moverla más al fondo, la Caja en Destino se comporta como caja y al moverla vuelve a ser destino.

El Suelo es el piso por defecto, no necesita casilleros intermedios porque por defecto al desplazarse una entidad se reemplaza el hueco que ocupaba por suelo normal

J_D	Jugador en Destino	Sokoban está parado sobre el Destino
C_D	Caja en Destino	La caja está en destino, funciona como caja, al desplazar la caja fuera vuelve a ser Casilla destino. Al tener todas las Casilla destino transformadas en Caja destinos ganas el nivel.
F_D	Caja frágil en Destino	Misma lógica, aplicada a la caja frágil
L-[CANAL] L_D	Caja llave en Destino	Misma lógica, aplicada a la caja llave
J_X-[CANAL]	Jugador en Cerrojo	Sokoban está parado sobre el Cerrojo
C_X-[CANAL]	Caja en Cerrojo	La caja está en el cerrojo, funciona como caja.
F_X-[CANAL]	Caja frágil en cerrojo	La caja frágil está en el cerrojo, funciona como caja.
L-[CANAL] X-[CANAL] L]	Caja llave en cerrojo	Caja llave sobre un cerrojo. Si coinciden los canales, el cerrojo se activa. Al desplazar la caja fuera, vuelve a ser Casillero cerrojo inactivo.

J_R	Jugador en suelo resbaladizo	Sokoban en suelo resbaladizo
C_R	Caja en suelo resbaladizo	Misma lógica
F_R	Caja frágil en suelo resbaladizo	Misma lógica
L-[CANA L]_R	Caja llave en suelo resbaladizo	Misma lógica
L-[CANA L1]+[CAN AL2]_X-[CANAL]- EXC	Caja llave multicanal en cerrojo exclusivo	Al colocarse, activa el canal del cerrojo y desactiva los demás canales de la llave

¿Por qué las intermedias?

Se nos ocurrieron otras 2 opciones a plantear a la hora de crear el mapa aparte de la “Intermedia”, las bautizamos como “Referencial” y “2 capas”

Referencial: La idea es que la matriz creacional solo detalle los suelos, y luego con un array aparte se detallan las entidades existentes (ej [J,C,C,F]) y luego con lógica se mueven esas entidades a las coordenadas designadas.

Problemáticas de este enfoque:

1. Viola el patrón Builder: El Builder ahora necesita lógica de negocio para vincular actores
2. Acoplamiento alto: El archivo de nivel necesita conocer coordenadas exactas
3. No escalable: Agregar 50 cajas = 50 líneas en la sección de actores

4. Propenso a errores: Sokoban: 2,2 puede estar sobre una Pared (inconsistencia)
5. Code Smell "Feature Envy": El Builder necesita saber demasiado del dominio.

Problemáticas de diseño:

1. Responsabilidad difusa: ¿Quién validaría que las coordenadas sean válidas?
2. Mantenimiento: Cambiar un mapa requiere recalcular todas las coordenadas a mano
3. Testeo: Difícil crear tests unitarios para el Builder
4. No visual: No se puede "ver" el mapa en el archivo

2 Capas: La idea es tener 2 matrices, una tiene los suelos, y la otra tiene los objetos que se desplazan (J,C,F,etc) o sea los objetos que van por encima.

Problemáticas de este enfoque:

1. Duplicación: Habría que mantener 2 matrices del mismo tamaño
2. Inconsistencia: Si se agregara una columna al suelo y se olvida hacerlo en objetos se crashea
3. No se justifica: La arquitectura de agregación (objetoEncima) ya resuelve esto
4. Dificulta el diseño: Se tendría que saltar entre dos secciones para entender un casillero
5. Es más complejo que la opción intermedia
6. Menos intuitivo al diseñar niveles
7. Requiere más validación al sincronizar capas

El pro es que si respeta factory method

Ideas de funcionalidades (2 a elegir)

- Portal: Permite a sokoban y las cajas teletransportarse al otro lado
- Pinchos: Las cajas frágiles que pasen por acá pierden puntos de vida o se rompen de una, sokoban tiene vida y va perdiendo cuando los pisa
- Caja que se destruye en colisión: Si toca una pared o otra caja se rompe
- Pisos que se rompan por cant de toques
- Paredes movedizas: Si tocas un botón la pared rota bloqueando un paso y abriendo otro: = o | |
- Gravedad: Al tocar un botón pisando o poniendo una caja encima, la gravedad se mueve para ese lado desplazando todas las cajas y a sokoban hasta ese extremo hasta que colisione con algo.
- Muros segmentados: Por ejemplo una puerta naranja y una azul, las cajas azules pasan por la azul pero no por la puerta naranja y viceversa.
- Cajas fusionables: Si una caja entra en contacto con otra igual se fusionan en un conjunto por ejemplo de dimensiones 2x1 si se juntan 2.

- Cintas transportadoras: Sokoban y las cajas dentro se desplazan en una dirección, lo hacen cada tantos segundos asique podes subir o bajar antes de que llegue al final de la cinta
- Vida: Sokoban tiene una vida anclada a sus movimientos, si te quedas sin movimientos disponibles tenes que reiniciar (combina con lo de pinchos, si tocas los pinchos perdes mas movimientos)
- Imanes: En el mapa hay imanes que lanzan un rayo atractor en línea recta hasta la caja, puedes bloquearlo poniendo una caja no metálica en el medio, moviendo una pared, desactivando el imán o esquivandolo, tambien podes usarlo a tu favor para cruzar una sección con la caja
- Caja bloqueada: Caja bloqueada que no puede desplazarse hasta que se active con la caja llave la cerradura

Requerimientos funcionales:

Número	Requerimiento	Descripción
RF 0	El mapa debe crearse en base a un TXT en el que se detalla el mapa	Se crea el mapa con los elementos dispuestos en la matriz del archivo de texto
RF 1	El usuario tiene que poder moverse	Sokoban se desplaza por el mapa
RF 2	El usuario debe colisionar con los objetos coleccionables	Sokoban se choca con cajas en fila o paredes
RF 3	El usuario debe desplazar cajas si es posible	Sokoban mueve la caja si esta "libre"
RF 4	El usuario debe poder sumar cajas hasta ganar	Si la cantidad X de cajas están ubicadas se gana el nivel
RF 5	Al completar el nivel el jugador debe poder pasar al siguiente nivel o repetir	Pantalla de nivel completado que permite reiniciar o jugar el siguiente y da los datos de score (Movimientos realizados, empujes efectuados, usos del botón undo, y puntaje obtenido)
RF 5.5	El usuario debe ver su puntaje obtenido	El puntaje se calcula considerando las variables de score
RF 6	El usuario debe tener un registro de time, movements, pushes y rollbacks	Panel inferior que muestra metadatos
RF 7	El usuario debe poder hacer rollback hasta 15 pasos atrás, 5 cada vez que presiona el botón	Snapshots de versiones anteriores del programa. Se guardan estados válidos (Si

	(máximo 3)	sokoban choca contra una pared 5 veces, esos movimientos no cuentan)
RF 8	El usuario debe resbalarse y desplazarse tanto él como las cajas en contacto con pisos resbaladizos	Sokoban y las cajas se resbalan hasta chocar con algo o que termine el piso
RF 9	El usuario debe poder activar cerrojos con llaves del mismo canal para abrir puertas bloqueadas. Comportamiento básico: - Cada caja llave, casillero cerrojo y muro cerrado pertenece a un canal identificado (ej: AZUL, ROJA, VERDE). - Al desplazar una caja llave sobre un cerrojo del mismo canal, el cerrojo se activa. - Cuando todos los cerrojos de un canal están activados, todas las puertas (muros cerrados) de ese canal se abren simultáneamente. - Si una caja llave se coloca sobre un cerrojo de diferente canal, la caja permanece sobre el cerrojo pero no lo activa. - Los canales son independientes entre sí. Ejemplos de escenarios - Simple: 1 llave AZUL → 1 cerrojo AZUL → 3 puertas azules - Múltiple: 2 llaves ROJAS → 2 cerrojos ROJOS → 1 puerta roja - Multicanal: 1 llave AZUL+ROJA → 1 cerrojo AZUL-EXC → toggle entre puertas azules/rojas - Combinado: 1 llave VERDE+AMARILLA → 2 cerrojos VERDES + 1 cerrojo AMARILLO-EXC	Características avanzadas: Llaves multicanal: - Una llave puede pertenecer a múltiples canales (ej: L-AZUL+ROJA). - Puede activar cerrojos de cualquiera de sus canales. - Permite resolver puzzles donde una única llave controla múltiples sistemas. Cerrojos exclusivos: - Un cerrojo en modo exclusivo (sufijo -EXC) desactiva los otros canales de una llave multicanal al ser activado. - Al colocar una llave L-AZUL+ROJA en X-AZUL-EXC: se abren puertas azules y se cierran puertas rojas. - Si luego se mueve a X-ROJA-EXC: se abren puertas rojas y se cierran puertas azules. - Crea mecánicas de "toggle" entre estados mutuamente excluyentes. Múltiples llaves por puerta: - Un canal puede requerir N llaves en N cerrojos para abrirse. - Ejemplo: Canal VERDE con 3 cerrojos → necesitas 3 llaves verdes en los 3 cerrojos simultáneamente para abrir las puertas verdes. - Las puertas permanecen cerradas hasta que todos los cerrojos del canal estén activados.
RF 10	El usuario debe transitar paredes abiertas	Cuando la pared bloqueada es abierta cambia el comportamiento a transitable

RF 11	El usuario debe poder ganar el nivel	El sistema debe evaluar constantemente si todas las cajas requeridas se encuentran sobre las casillas de destino. Al cumplirse, se debe habilitar el paso al siguiente nivel.
RF 12	El usuario debe poder reiniciar el nivel	El jugador debe disponer de una opción para recargar el nivel actual a su estado inicial original en cualquier momento de la partida.

Requerimientos no funcionales:

Número	Requerimiento	Descripción
RNF 1	Usabilidad	El programa deber ser amigable para el usuario y que sepa como interactuar con el
RNF 2	Patrón arquitectónico	El código fuente del proyecto debe estar estructurado y dividido estrictamente bajo el patrón arquitectónico Modelo-Vista-Controlador (MVC).
RNF 3	Lectura de niveles	El juego debe inicializar sus escenarios leyendo archivos externos de formato de texto plano (.txt), mapeando cada carácter a una entidad del dominio.

Métodos y estrategias:

Generación de mapas:

La solución es implementar una combinación de **Factory Method** y **Singleton**. La lógica consistiría en tener una **clase gestora o directora** que recorra la matriz del archivo de texto por conjuntos de caracteres separados por comas (,). Por cada conjunto de símbolos que lea se lo pasa a una fábrica de entidades, así evitando los if-else hardcoded por entidad. Esta fábrica evalúa los caracteres y se encarga de instanciar y devolver el objeto concreto que corresponda, ya sea una caja frágil, un piso resbaladizo o una llave, para que el gestor lo ubique en las coordenadas correspondientes del tablero. Para optimizar esto, la fábrica puede ser un Singleton (FabricaDeEntidades), asegurando que exista una única instancia encargada de crear elementos durante toda la carga del nivel, ya que de no usarse, si el tablero fuera un 20x20, tendrían que haber 400 fábricas.

Generación y construcción de mapas:

Para la lectura y el armado de los escenarios se decidió implementar una arquitectura creacional combinando los patrones Factory Method, Singleton y Builder de forma integrada.

En primer lugar, dado que los niveles se inicializan a partir de archivos de texto plano, se utiliza el patrón Builder mediante la clase Constructor Tablero para separar el proceso complejo de construcción del tablero de su representación final. Este constructor se encarga de recibir el archivo línea por línea, definir las dimensiones de la grilla y orquestar el posicionamiento paso a paso de cada objeto en sus coordenadas correspondientes.

En segundo lugar, para evitar bloques de código acoplados dentro del lector, el proceso de instancia específico de cada entidad se delega a una clase FabricaDeEntidades que aplica el patrón Factory Method parametrizado. Esta fábrica recibe el carácter leído del archivo de texto y se encarga de mapear y retornar el objeto concreto correspondiente del modelo, como una pared, un suelo o el jugador.

Finalmente, para optimizar el uso de recursos de memoria durante el ciclo de vida del juego, la fábrica se implementa bajo el patrón Singleton, garantizando la existencia de una única instancia global encargada de centralizar la creación de todas las entidades del mapa.

Jerarquía de Clases y Arquitectura del Modelo

Para dar soporte a la creación dinámica de mapas mediante los patrones **Builder**, **Factory Method** y **Singleton**, el dominio del juego se dividirá en una estructura jerárquica basada en abstracciones.

Clase base: Entidad

Es la clase base de todos los elementos del tablero, tiene propiedades abstractas que comprenden a todas las entidades.

Atributos:

- x: int (Posicion en el eje X)
- y: int (Posicion en el eje Y)
- tipoRepresentativo: String (Caracter/es iniciales que mapea la fábrica)

Métodos abstractos:

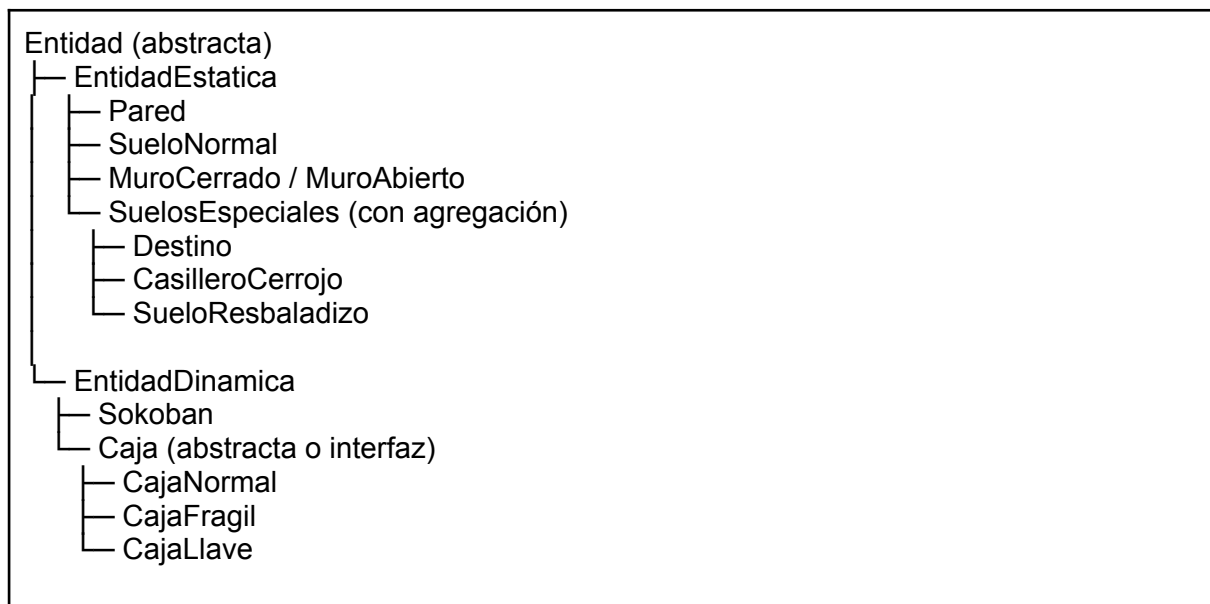
- esTransitable(): boolean (Define si Sokoban o una caja pueden ocupar el espacio)
- esEmpujable(): boolean (Define si reacciona a fuerzas externas)

Clasificación de Entidades

Las piezas del juego se agrupan en dos grandes familias heredadas de Entidad:

1. Entidades dinámicas: Representan los elementos que poseen comportamiento de desplazamiento y alteran su posición en la matriz.
 - Sokoban: Personaje jugable principal.
 - CajaNormal: Caja estándar que debe ser empujada hacia los destinos.
 - CajaFragil: Caja con límite de desplazamientos antes de romperse.
 - CajaLlave: Caja utilizada para activar las casillas candado.
2. Entidades Estáticas: Representan objetos fijos del escenario
 - Pared: Bloque estático infranqueable.
 - SueloNormal: Superficie básica transitable por defecto.
 - MuroCerrado / MuroAbierto: Barreras temporales gobernadas por el estado del cerrojo.

Gráficamente se vería así



Implementación de casilleros intermedios

Para evitar tener un montón de subclases intermedias, los suelos especiales se modelan como estructuras contenedoras dinámicas. Las clases Destino, CasilleroCerrojo y SueloResbaladizo heredan de entidades estáticas, pero incluyen un atributo de agregación.

- Atributo clave: Entidad Dinámica objetoEncima (null por defecto)

Mecánica de estados en tiempo de ejecución:

- Si objetoEncima == null, el casillero actúa y se renderiza en su estado base libre (D, X o R).

- Si Sokoban o una caja entran en sus coordenadas, el controlador asigna dicho objeto al atributo objetoEncima.
- El método esTransitable() del casillero evaluará dinámicamente si hay algo encima bloqueando el paso, permitiendo al sistema mutar de comportamiento sin perder la información del suelo que se encuentra debajo. Cuando el objeto se retira, objetoEncima vuelve a ser null y el suelo original reaparece.

Sistema de canales para llaves/cerrojos/muros

Para soportar múltiples sistemas independientes de llaves-cerrojos-puertas con comportamientos avanzados, se implementa un ****sistema de identificadores por canal con soporte multicanal****:

1. Identificación: Cada `CajaLlave` tiene una lista de canales (`List<String> canales`), mientras que `CasilleroCerrojo` y `MuroCerrado` tienen un único canal (`String idCanal`).
2. Activación condicional: Un cerrojo se activa si la llave colocada encima pertenece a su canal. La validación se realiza mediante `llave.perteneceACanal(cerrojo.getIdCanal())`.
3. Modo exclusivo: Los cerrojos pueden tener un flag `modoExclusivo`. Al activarse con una llave multicanal, desactivan automáticamente los otros canales de esa llave mediante `Tablero.desactivarOtrosCanales()`.
4. Gestión por el Tablero: El tablero mantiene dos mapas:
 - `Map<String, List<CasilleroCerrojo>> cerrojosPorCanal`
 - `Map<String, List<MuroCerrado>> murosPorCanal`
5. Apertura de puertas: Cuando se detecta que ****todos**** los cerrojos de un canal están activados, se ejecuta `abrirMurosDeCanal(canal)`.
6. Cierre de puertas: En cerrojos exclusivos, al desactivar un canal se ejecuta `cerrarMurosDeCanal(canal)`.
7. Comportamiento permisivo: Si una llave se coloca en un cerrojo de canal no coincidente, el movimiento es válido pero no produce activación.
8. Codificación en archivos:
 - Llaves: `L-CANAL` o `L-CANAL1+CANAL2+...`
 - Cerrojos: `X-CANAL` o `X-CANAL-EXC`
 - Muros: `M-CANAL`